Why a Circular Queue Prevents Wasted Array Space After Dequeues

Course: CpE211 Lab 5 · **Student:** Kane Paul A. Mari (BSCpE-2) · **Instructor:** Sir Jamie Eduardo Rosal · **Date:** 10/24/2026

Question

> "Why does a circular queue prevent wasted array space after dequeues?"

Answer (student-verified)

In a **plain (linear) queue**, every dequeue advances *front*, so the slots it leaves behind become empty but unreachable — *rear* only moves forward until it hits the last index of the array. As a result, even if you dequeue everything inside, an enqueue can still overflow: "false overflow" caused by stranded freed slots rather than a genuinely full array.

A **circular queue** fixes this by treating the array as a circle: when *rear* or *front* reaches the end, the wrap `(index + 1) % capacity` sends it back to index 0. Later enqueues therefore land in the very slots that dequeues freed, so every cell of the fixed-size array stays recyclable — no shifting, no reallocation, and dequeue remains O(1).

Worked trace (capacity 5)

```
LINEAR QUEUE
  enqueue A B C D E    [A B C D E]   fill slots 0..4; front stays 0
  dequeue A, dequeue B  ->  slots 0 and 1 are freed but lost to enqueues
  enqueue F             OVERFLOW - rear is parked at slot 4
                        while 2 slots sit EMPTY at the front  (waste)

CIRCULAR QUEUE
  enqueue A B C D E                   front = 0, rear = 4, count = 5 (full)
  dequeue A   ( front 0 -> 1 )   dequeue B   ( front 1 -> 2 )
                                       -> slots 0 and 1 freed
  enqueue F   rear = (4 + 1) % 5 = 0    F lands in slot 0  (freed by A)
  enqueue G   rear = (0 + 1) % 5 = 1    G lands in slot 1  (freed by B)
  state:  [F G C D E]  front = 2   ->  all 5 slots re-busy, ZERO waste
```

Extra remarks

- The *count* field distinguishes empty (`count = 0`) from full (`count = capacity`) — in a ring, the indices can look identical in both states (the same "front just behind rear" configuration).
- The modulo operator is the remainder trick: 5 % 5 = 0, so index 5 folds back onto index 0, making the linear array behave like a ring.